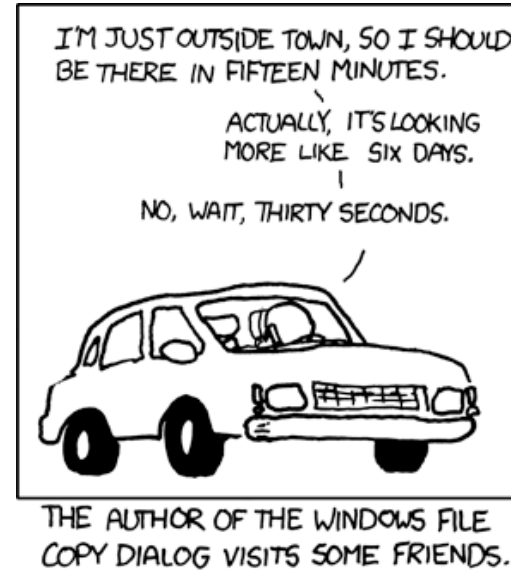
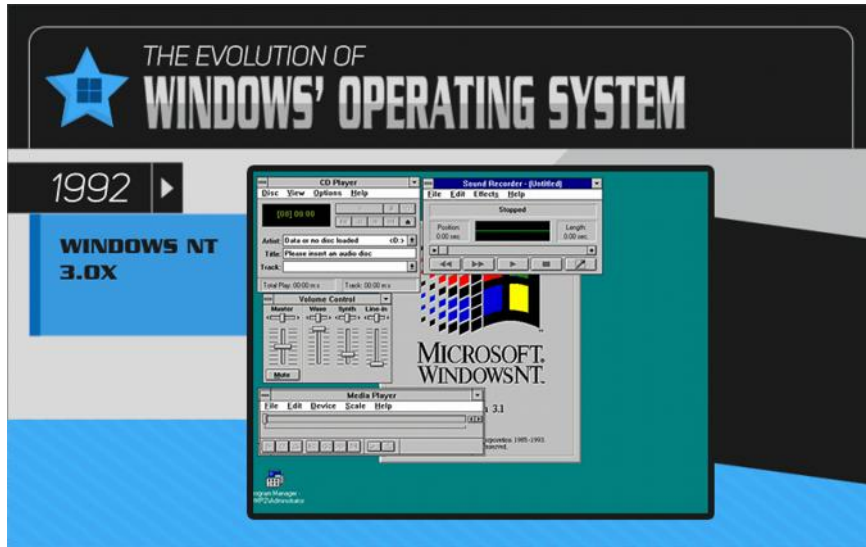
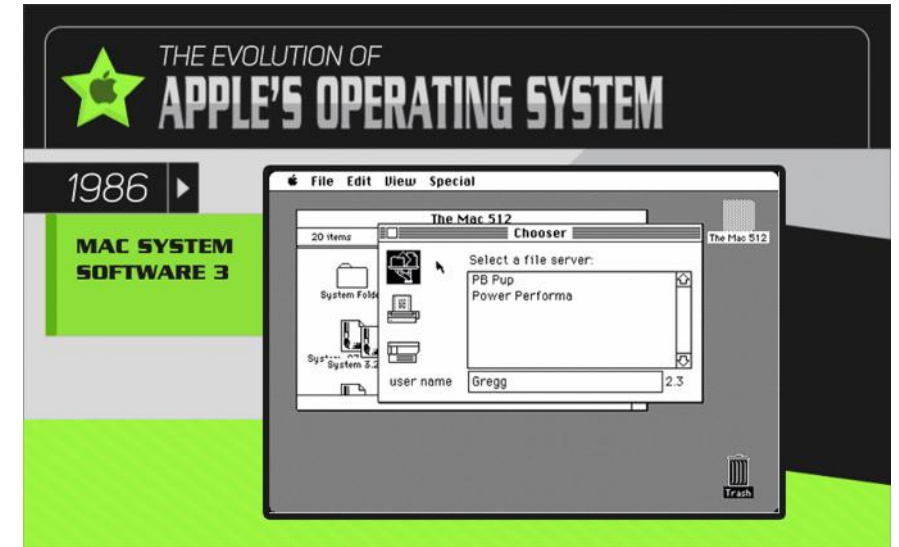




<https://xkcd.com/612>



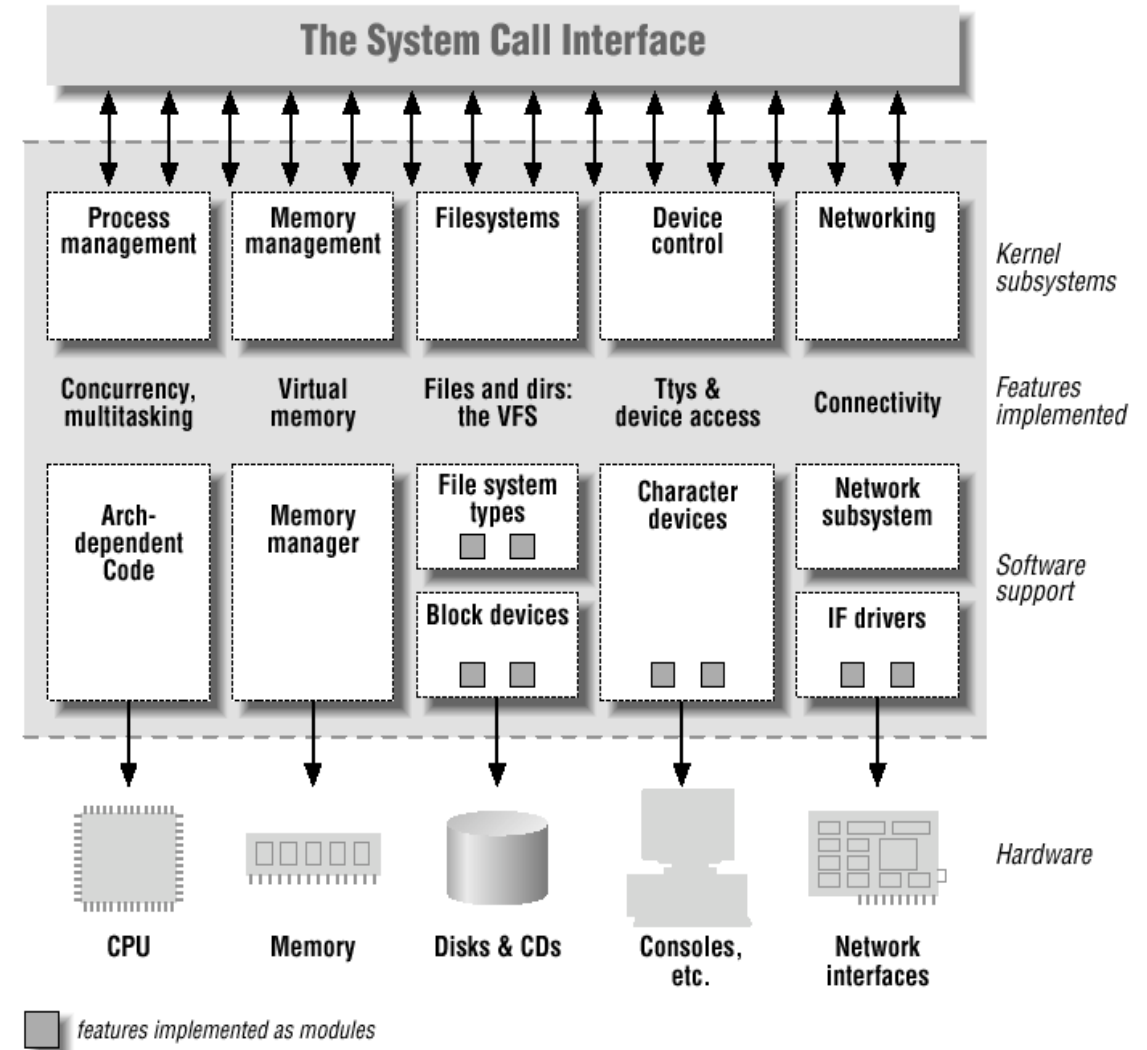
Special Topic - RTOS

(3B) *Operating Systems Concepts*

An Operating System (OS)

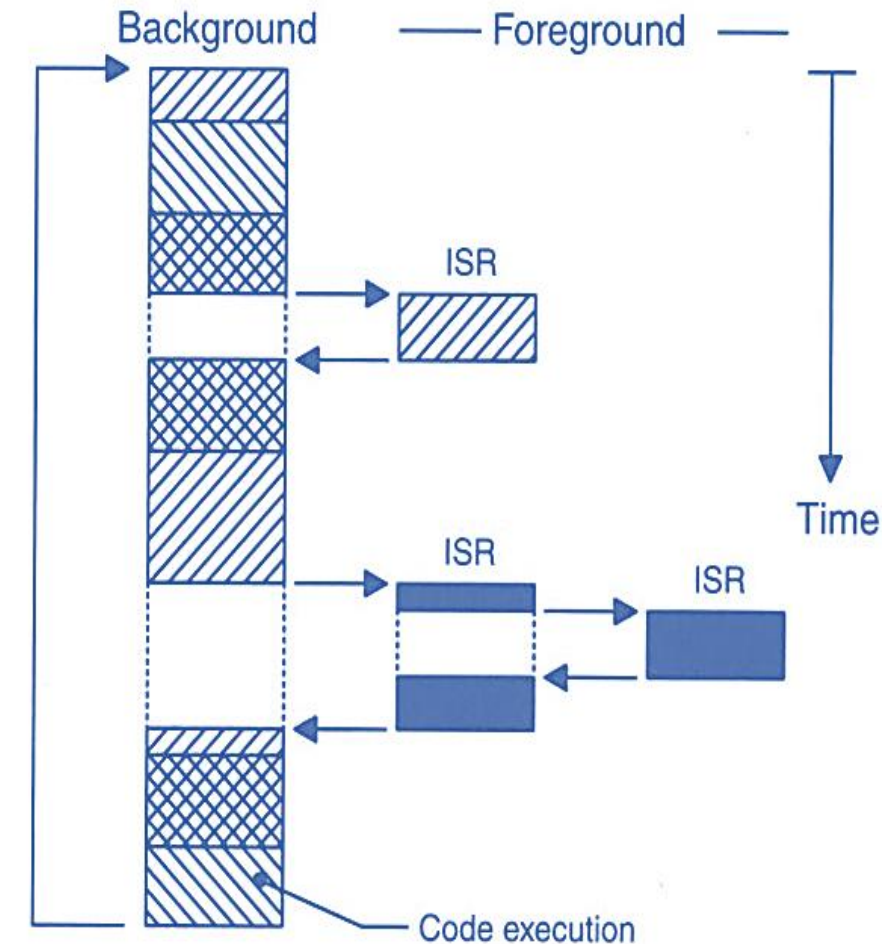
The Operating System (OS) provides:

- Process Management
- Memory Management
- Hardware Abstraction (Device Control)
- File Systems Support
- Networking
- A system call interface



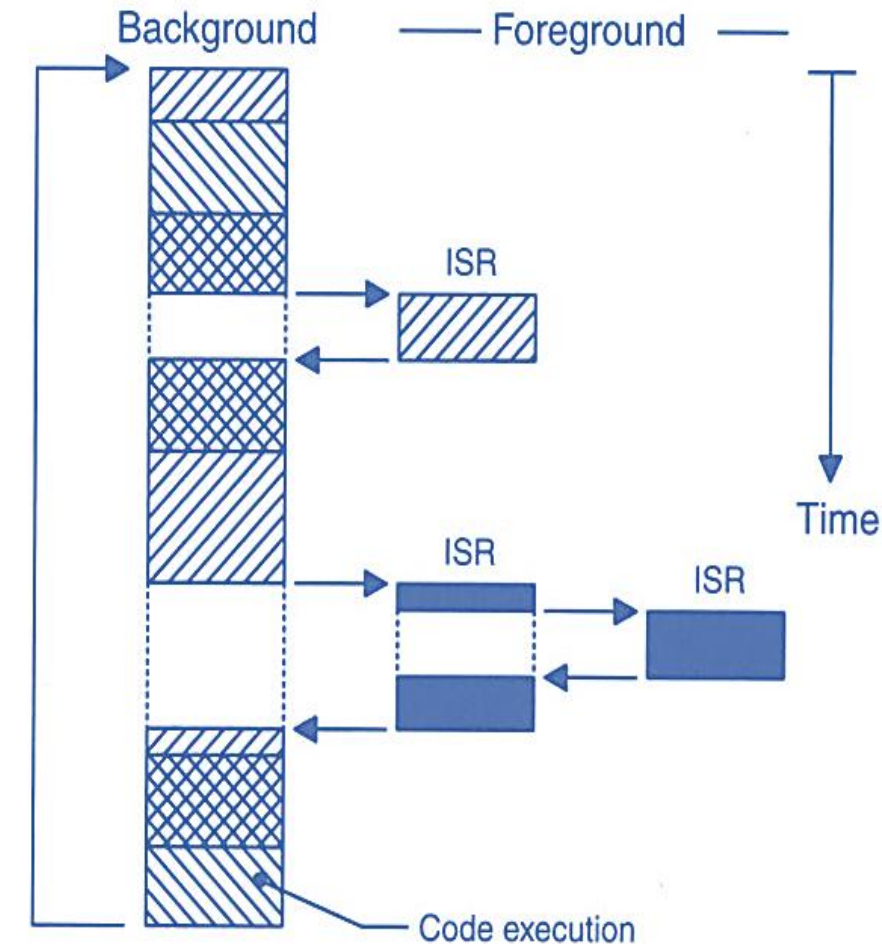
OS Concepts : Foreground/Background Systems

- Small systems of low complexity are generally referred to as **foreground/background systems** or **super-loops**.
- The application consists of an infinite loop that calls modules (functions) to perform specific functions (background). Interrupt service routines handle asynchronous events (foreground).
- Foreground is referred to as **interrupt-level**, while background is commonly referred to as **task-level**.



OS Concepts : Foreground/Background Systems

- Critical operations are performed at the interrupt-level as these must be dealt with in a timely manner.
- Data provided by an interrupt level task to a background task, is not processed by that task until it gets a turn to execute (task-level response).
- Microwave ovens, toys, telephones, etc utilize this type of OS



OS Concepts : Critical Sections (of Code)

- Also known as *critical region*.
- Once a critical sections begins it can not be interrupted, it must be completed.
- **Disabling interrupts** is one sure way of ensuring that the critical code completes uninterrupted. Once the code is complete interrupts can be enabled once more.
- Critical Sections also relate to areas of code in which commonly shared data is modified, and the code must complete the modification prior to any other process using it.

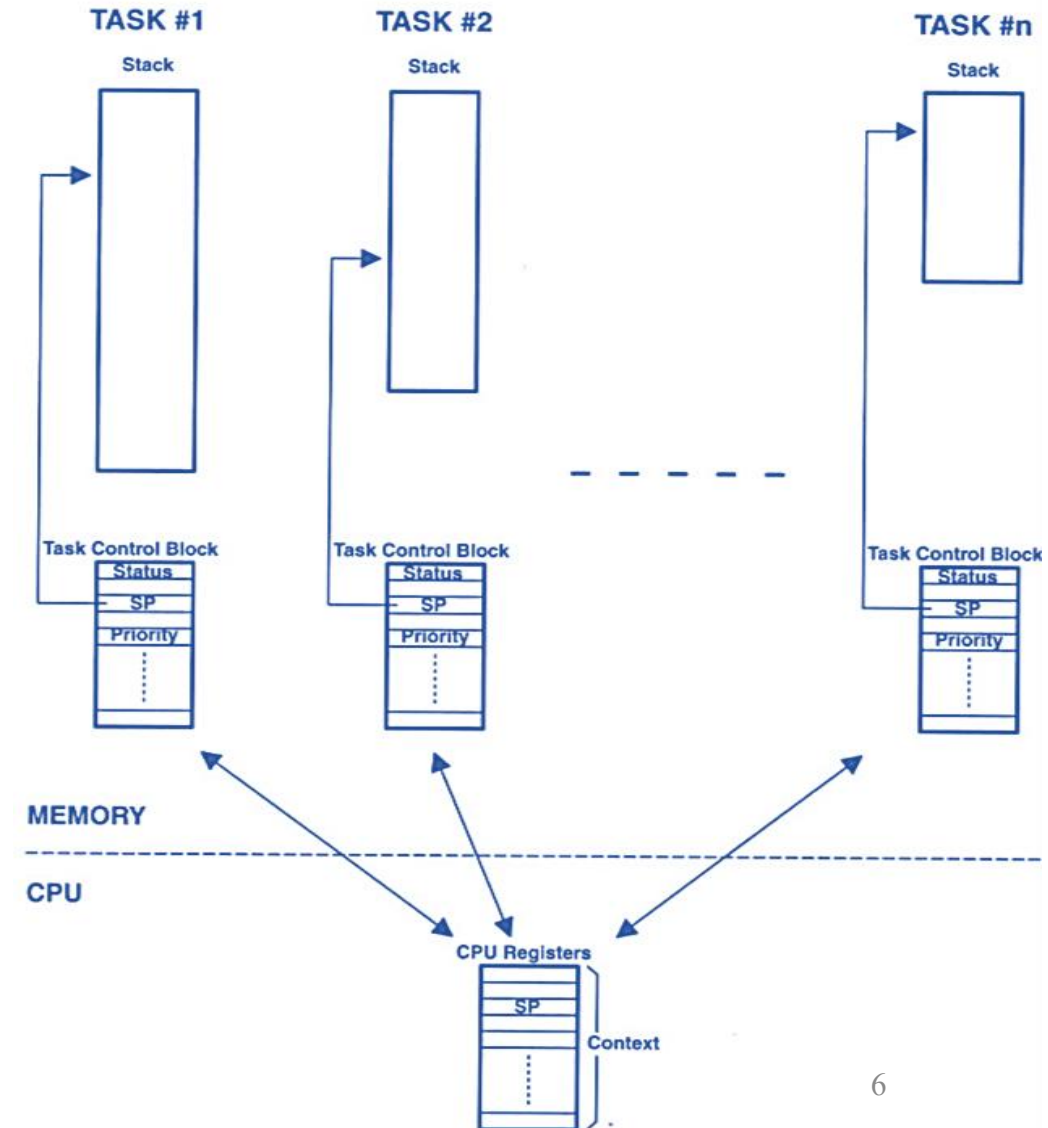
OS Concepts : Resources & Tasks

Resources

- A resource is any item that can be used by a task.
- These include memory, CPU, I/O (peripheral).

Tasks

- Also known as **threads**.
- A task believes it has the CPU all to itself.
- Each task has a priority, its own set of CPU registers, and its own stack.



OS Concept : Tasks

- Each task is effectively an infinite loop that can be either:

Dormant

- Resides in memory but not yet available to the kernel.

Ready

- Can execute however its priority is lower than the task currently running.

Running

- Has control of the CPU.

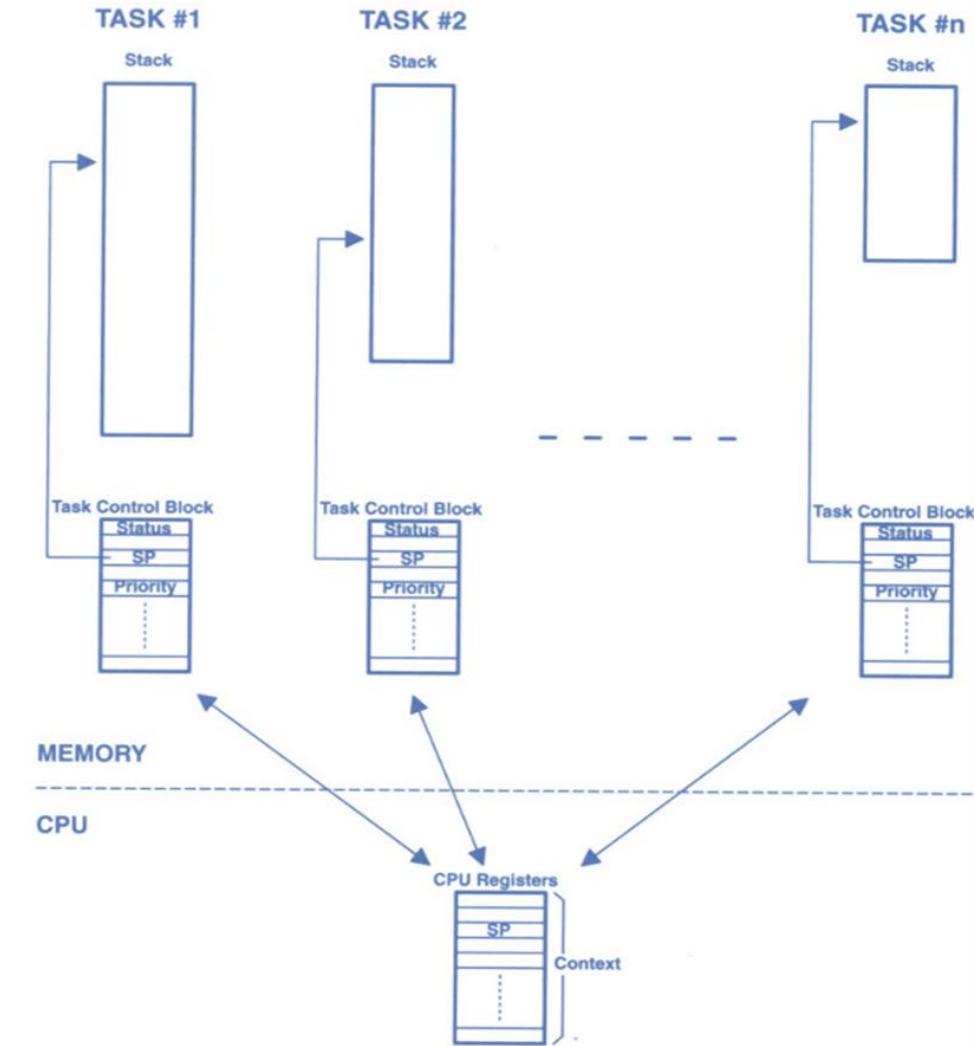
Waiting (for an event),

- Requires an event to take place first.

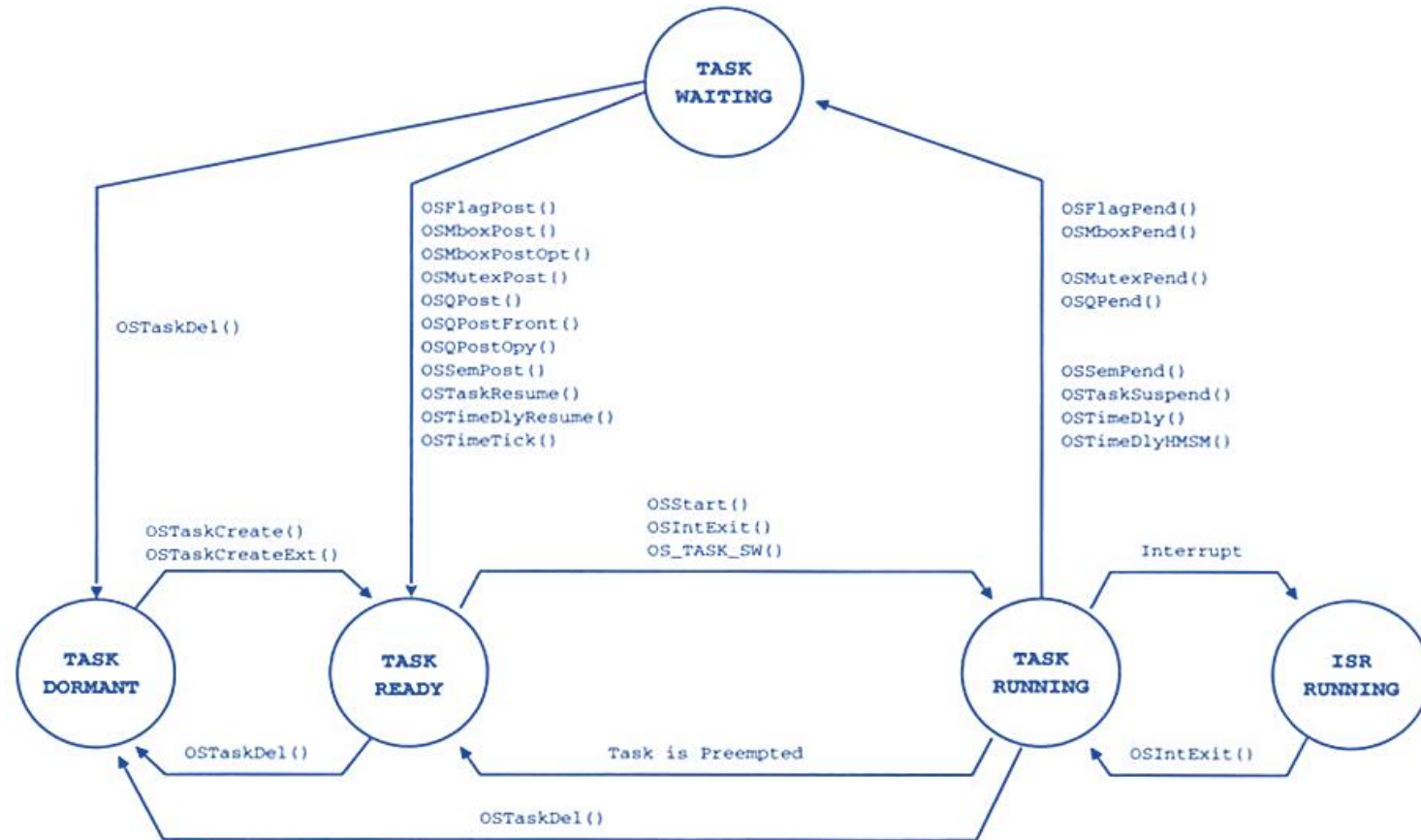
or

ISR (Interrupted).

- Services an event.



OS Concepts : Task States



OS Concepts : Context Switch

- When the CPU decides to run another task it stores the current CPU context in the current tasks context storage area.
- After the operation is complete the new tasks context is restored from its storage area onto the CPU and the new task resumes execution.
- This process is commonly referred to as **context switching** or **task switching**.
- Context switching adds overhead to the application. The time required to perform a context switch depends on the number of registers within the CPU as they all must be saved and restored.

OS Concepts : Kernels

- The kernel is part of a multitasking system responsible for management of tasks and communication between tasks.
- The fundamental service provided by the kernel is **context switching**.
- The kernel adds overhead to the system as it also needs to be executed by the processor.
- The amount of overhead depends on how often you use the services provided by the kernel.
- The kernel requires space for:
 - Its own code,
 - Its own data structures, and
 - A task structure for each task.
- The kernel introduces a strain on the memory resources of the system.

OS Concepts : Reentrant Functions

- A reentrant function can be used by more than one task at a time without the fear of data corruption.
- A reentrant function can be interrupted at any time and continued later with no effects on its outcome.
- Reentrant functions use only local variables or protected global variables.

Non-Reentrant

```
int Temp;  
...  
void swap (int *x, int *y){  
    Temp=*x;  
    *x=*y;  
    *y=Temp;  
}
```

- Since this function uses a global variable all instances of the function will access the same global variable. Hence there is a big probability of data corruption.

Reentrant

```
void strcpy (char *dest, char *src){  
    while (*dest++ = *src++){  
        ...  
    }  
    *dest=NULL;  
}
```

- Since this function's input parameters are placed on the stack, this function can be run simultaneously without fear of data corruption.

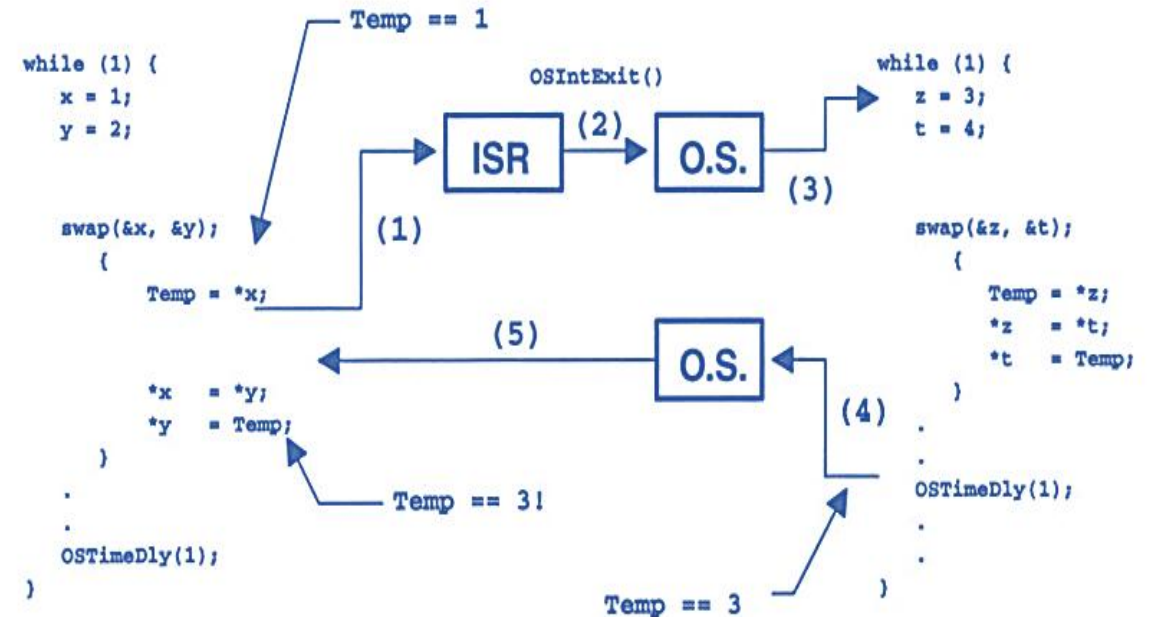
OS Concepts : Non-Reentrant Function Example

When swap is interrupted Temp contains 1.

- i. An ISR reads a higher priority task.
- ii. The kernel triggers a context swap to the higher priority task that also uses the swap function.
- iii. The high priority task finishes leaving the value of Temp as 3.
- iv. The lower priority task runs using the new value of Temp not the value it originally stored in Temp.

LOW PRIORITY TASK

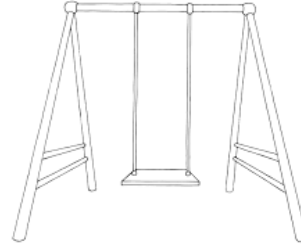
HIGH PRIORITY TASK



Analogy – Mutual Exclusion



Flag=0



Flag=0



Flag=1

1 Minute Later



Flag=1



Busy

1 Minute Later



Flag=0



FREE



Flag=1

Mutual Exclusion : (Mutex Concepts)

- The easiest way for tasks to communicate with each other is through shared data structures.
- Task intercommunication is easy when the tasks exist in the same address and can reference elements such as global variables, pointers, linked lists, etc.
- Even though sharing data simplifies the exchange of data, care must be taken to ensure that each task has exclusive access to the data to avoid contention and data corruption.
- Common methods of gaining exclusive access to shared resources are:
 - Disabling and enabling interrupts,
 - Flags,
 - Scheduling Techniques, and
 - Semaphores

Mutual Exclusion : Disable/Enable Interrupts

- The easiest and fastest way to gain exclusive access to a shared resource is by disabling and enabling interrupts.
- While the interrupts are disabled it is impossible for the kernel to preempt another task.
- Care must be taken to ensure interrupts are not disabled for too long. Doing so will effect the ***interrupt latency*** of your system.
- You should use this method if you are storing to only a few variables.
- This method is the only way of sharing data with an ISR.

Mutual Exclusion : Flags

- Flags also know as *Test-and-Set Operations*.
- Used in situations where a kernel is not being used.
- Two tasks could access a shared resource by first checking the condition of a global flag, if **set** the resource is being used else the resource is free to be used by the task.
- The first function would set a flag to prevent any other task from accessing the same shared resource.
- The flag must either be set by the processor itself or interrupts must be disabled when testing and then setting the flag.
- This will ensure that no other task sets the flag prior to the operation being completed.

Mutual Exclusion : Scheduler Techniques

- If your task is not sharing data with an ISR then you can disable and enable scheduling.
- By disabling scheduling you do not disable interrupts, so an ISR can still interrupt your task.
- However, on completion of the ISR, control of the CPU will be returned to the interrupted task even if a higher priority task is now ready to run.
- Because the kernel returns to the interrupted task this makes the preempted kernel function in a similar fashion to a non-preempted kernel.
- When scheduling is enabled once more, control of the CPU will be given to a higher priority task if one has been made ready while scheduling was disabled.

Mutual Exclusion : Semaphore Concept

- Semaphores for computing were invented during the mid-1960s.
- Semaphores are a protocol mechanism offered by most multitasking kernels.
- Semaphores are used to:
 - Control access to a shared resource (mutual exclusion),
 - Signal the occurrence of an event, and
 - Allow two tasks to synchronize their activities.
- A semaphore is a key that a task acquires in order to continue execution. If the semaphore is already in use then the requesting task suspends until the semaphore is free.
- There exists two types of semaphores:
 - Binary Semaphores**
 - Binary semaphores can only take the value 0 or 1.
 - Counting Semaphores**
 - Counting Semaphores allow any value between 0 and 255, 65536,...

Mutual Exclusion : Semaphore

- Generally only three operations can be performed on a semaphore:
INITIALIZE (CREATE),
WAIT (PEND) and,
SIGNAL (POST)
- A task desiring the semaphore performs a WAIT operation. If the semaphore is available (the semaphore value is greater than 0) then the semaphore is decremented and the task continues execution.
- 0 indicates used, non-zero indicates available

Mutual Exclusion : Semaphore

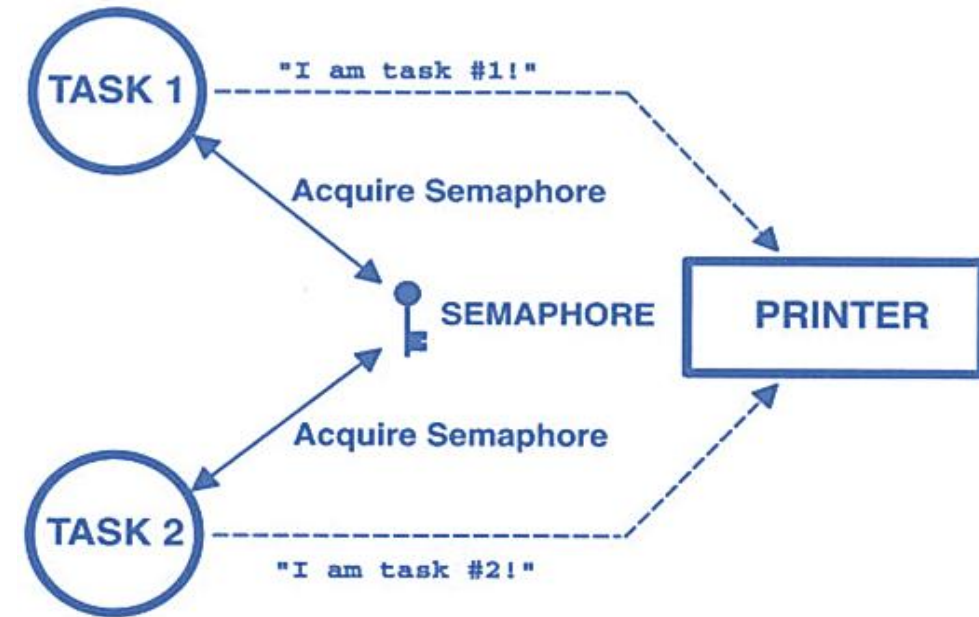
- If the semaphore's value was 0 when the WAIT operation was performed, which means the Task was placed on a waiting list.
- Most kernels allow you to specify a timeout, that once it is reached the requesting task is made ready to run and an error code is returned to the caller.
- A Task releases the semaphore by performing a SIGNAL operation. If no task is waiting for the semaphore, the semaphore value is simply incremented.
- If a Task is waiting then the semaphore is not incremented, it is simply given to one of the waiting tasks.

Mutual Exclusion : Semaphore

- Depending on the kernel, the task that receives the semaphore is either:
 - The **highest priority** task waiting for the semaphore or
 - The **first task** that requested the semaphore (first in, first out).
- Generally kernels allow you to choose which scheme it should adopt.
- If the readied task has a higher priority than the one releasing the semaphore, a context switch occurs (with preemptive kernels) and the higher priority task resumes execution.
- The lower priority task is suspended until the higher priority task completes.

Mutual Exclusion : Semaphore

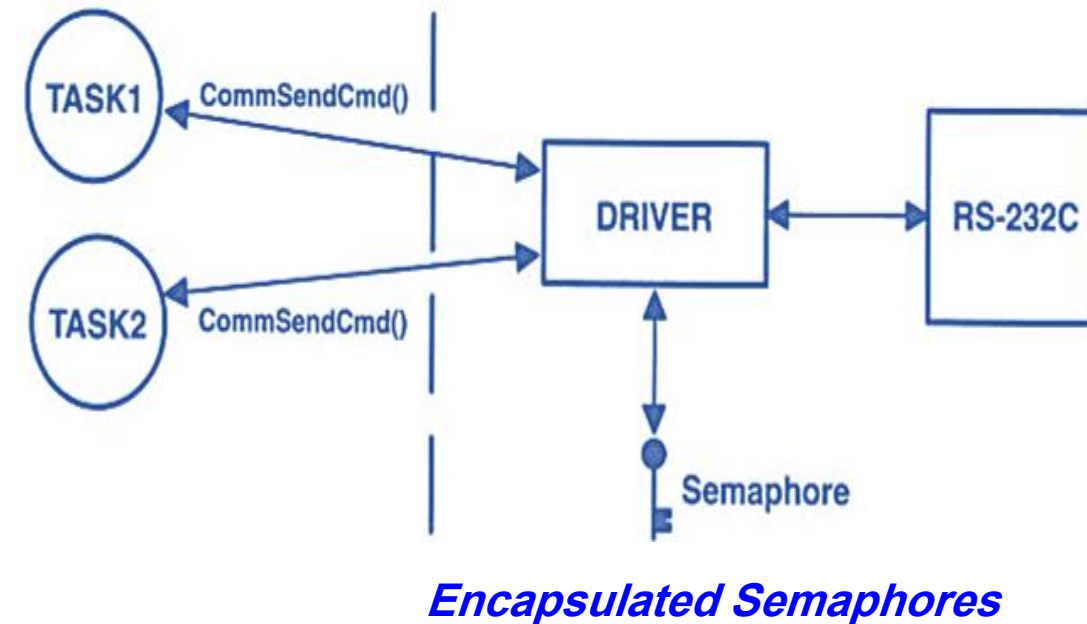
- Semaphores are especially useful when tasks share I/O peripherals.
- Using the printer as an example, can you imagine the mess that would occur if two tasks tried to print at the same time?
- In this case using a binary semaphore would ensure that only one task had access to the printer at any given time.
- Each task must be aware of the existence of the semaphore.



**Semaphores
with shared
resources**

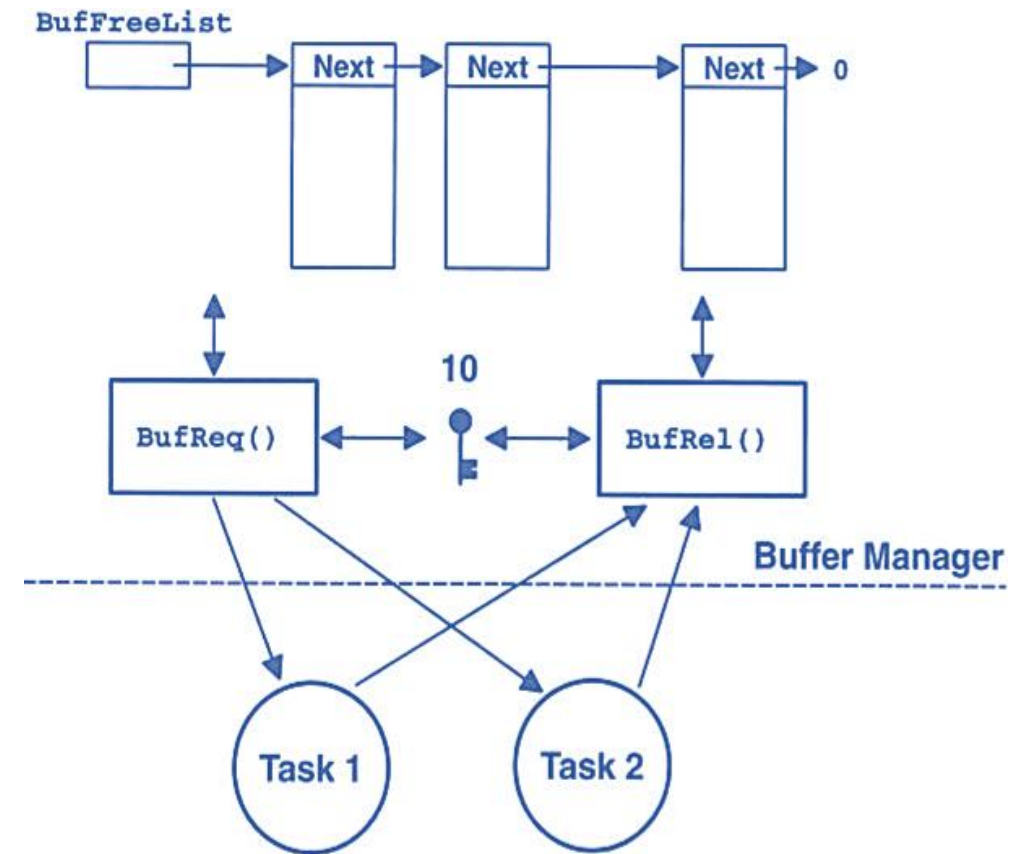
Mutual Exclusion : Semaphore

- In some cases, it is better to encapsulate the semaphore.
- Each task wouldn't need to know it is acquiring a semaphore when accessing the resource.
- Eg The function **CommSendCmd()** would take care of acquiring and releasing the semaphore.
- Therefore, any task that wants to communicate to this device would have to do so via this common system call.
- If another task uses the function **CommSendCmd()** while it is currently being used by another task, the hidden semaphore will suspend the second task.



Mutual Exclusion : Semaphores

- A counting semaphore is used when a resource can be used by more than one task at the same time.
- A counting semaphore can be used in the management of a buffer pool.
- For example, if a buffer pool originally contained 10 buffers.
- A buffer is acquired by calling the function `BufReq()`, and released by calling the function `BufRel()`.
- Each time `BufReq()` is called a semaphore is acquired and the semaphore decrements.
- Each time `BufRel()` is called the semaphore is released and the semaphore is incremented.



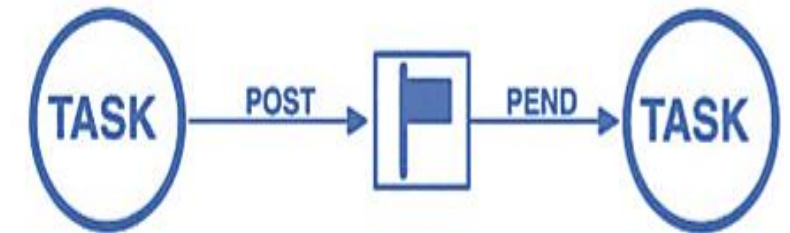
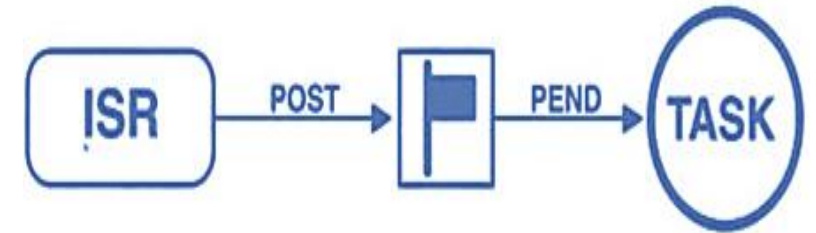
Counting Semaphores

Mutual Exclusion : Deadlock

- Also known as deadly embrace.
- A deadlock occurs when two tasks are unknowingly waiting for resources held by the other.
- For example
 - Task 1 is using resource 1 and waiting to get hold of resource 2.
 - Task 2 is using resource 2 and waiting to use resource 1.
- Neither of these two tasks can continue, they are **deadlocked!!**
- The easiest way to ensure this doesn't happen is:
 - Each task acquires a lock for all resources it requires at the start before proceeding,
 - Acquires resources in the same order, and
 - Release resources in the reverse order.
- Most kernels allow you to specify a timeout when attempting to acquire a semaphore.
- Deadlocks generally occur in large multitasking systems and are unusual in embedded systems.

Mutual Exclusion : Synchronisation

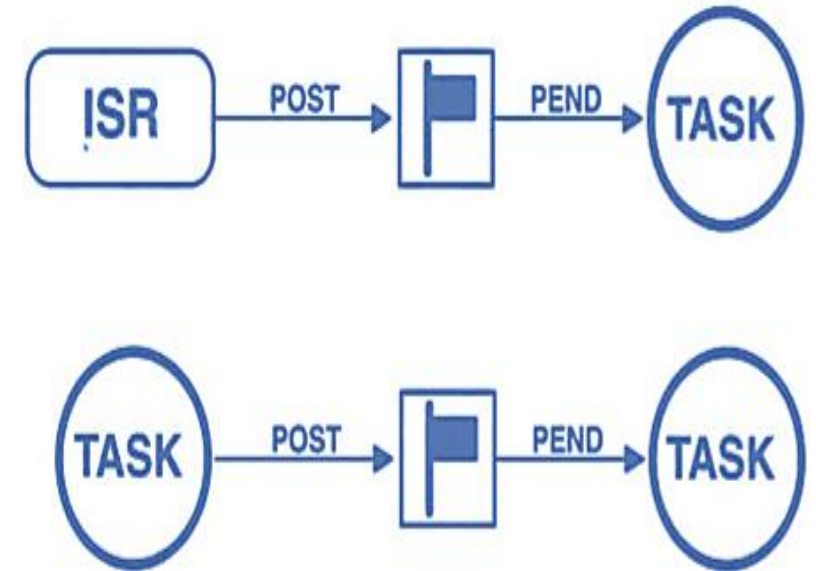
- A task can be synchronized with an ISR or another Task (when no data is being transferred) by the use of a semaphore.
- The semaphore is used to flag the occurrence of an event instead of acquiring exclusive access to a resource.



Task Synchronisation (Unilateral Rendezvous).

Mutual Exclusion : Synchronisation

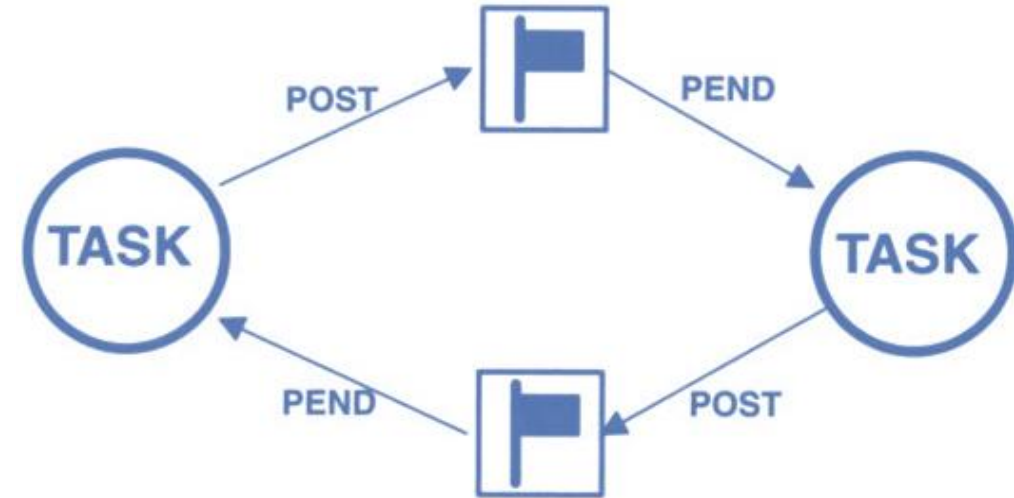
- When used as a synchronization mechanism, the semaphore is initialized to '0' (unilateral rendezvous).
- A task initiates an I/O operation and waits for the semaphore. Once the operation is complete an ISR or other task can release the semaphore allowing the original task to continue.



Task Synchronisation (Unilateral Rendezvous).

Mutual Exclusion : Synchronisation

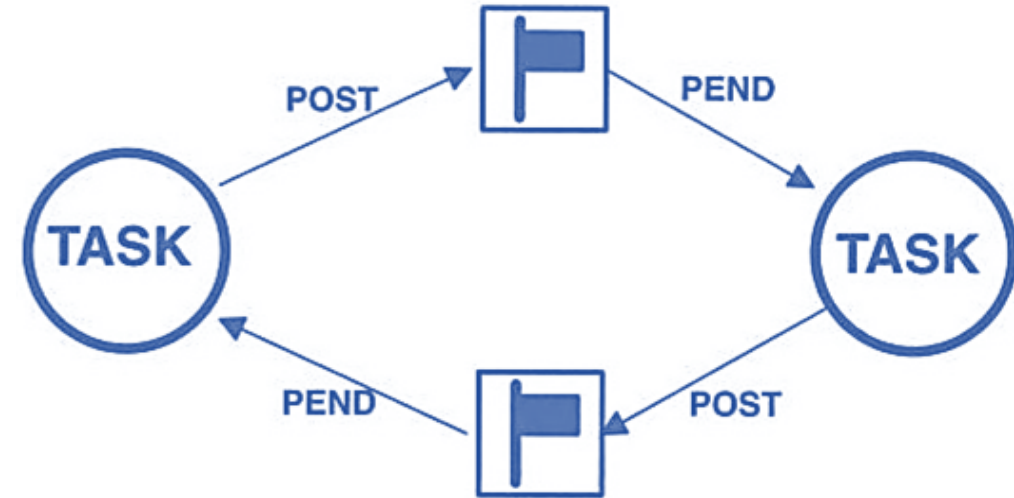
- If the kernel supports counting semaphores, the semaphore can then accumulate events that have not yet occurred.
- More than one task can be waiting for an event to occur.
- The kernel signals the occurrence of the event either to the highest priority task or the first task waiting for the event.



Task Synchronization (Bilateral Rendezvous).

Mutual Exclusion : Synchronisation

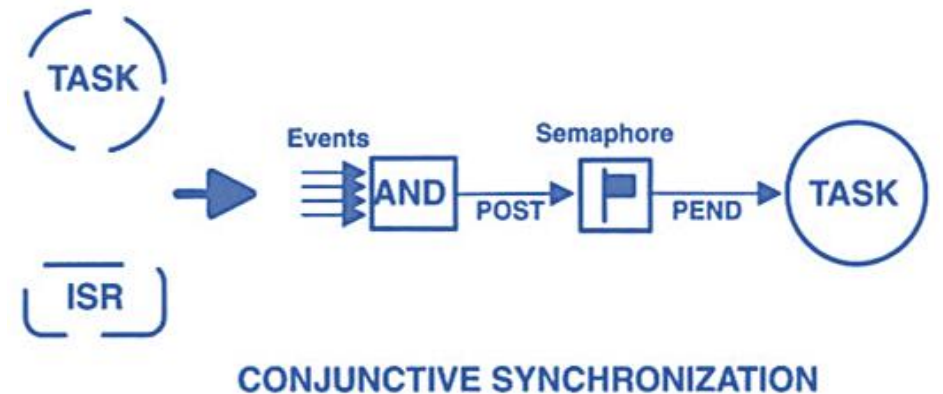
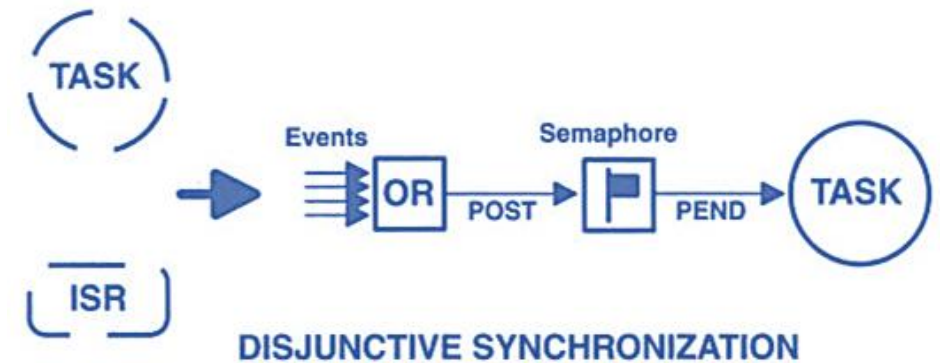
- Depending on the application more than one ISR or task can signal the occurrence of the event.
- Two tasks can synchronize their activities by using two semaphores a procedure commonly referred to as ***bilateral rendezvous***.
- Bilateral rendezvous differs from unilateral as both tasks must synchronize with each other before continuing.



Task Synchronization (Bilateral Rendezvous).

Mutual Exclusion : Event Flags

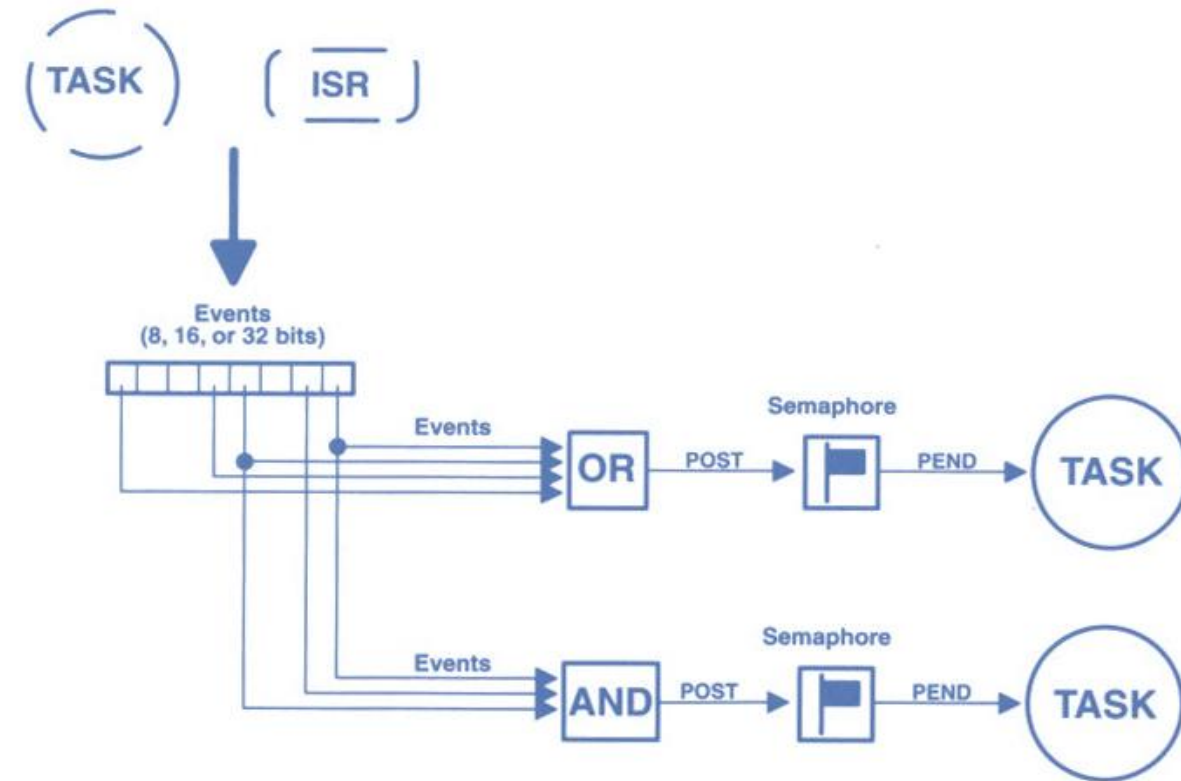
- An event flag is used when a task needs to synchronize with the occurrence of multiple events.
- In **disjunctive synchronization**, the task can be synchronized when any of the events take place (OR) or the task can be synchronized when all of the events have taken place (AND).



Task Synchronisation (Event Flags)

Mutual Exclusion : Event Flags

- Common events can be used to synchronize multiple tasks.
- Events are generally grouped, depending on the kernel each group consists of 8, 16, or 32 events, each represented by a bit.
- Tasks or ISR can set or clear any event within a group. A task is resumed once all the events it requires are satisfied.
- Kernels provide services to:
 - set event flags,
 - clear event flags, and
 - wait for event flags.



Synchronisation (Event Flags)

Scheduler: Types

- The scheduler is also known as the dispatcher.
- Responsible for deciding which task runs next.
- Each task is assigned a priority based on its importance.
- Control of the CPU is **always** given to the task with the highest priority that is ready to run.
- When the highest priority task gets the CPU depends on whether the scheduler is preemptive or non-preemptive.

Non-Preemptive:

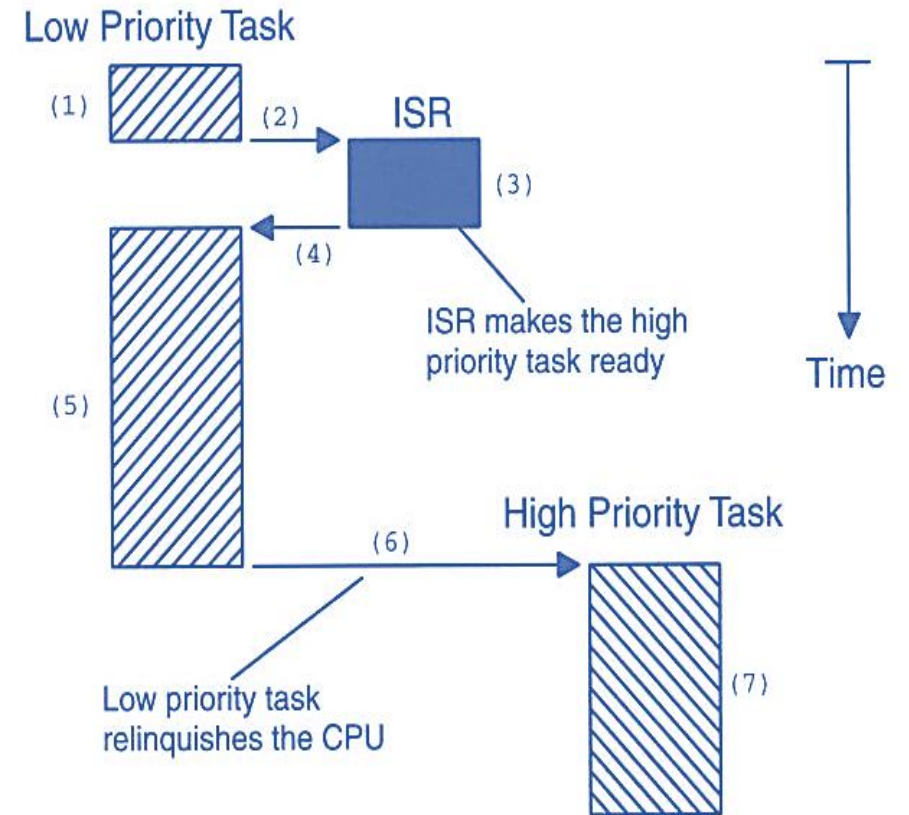
When a higher priority task is ready to run the current running task continues to run to completion before giving control to the higher priority task.

Preemptive:

When a higher priority task is ready to run the current running task is ***preempted*** (suspended) and the higher priority task is run to completion (unless interrupted by an even higher priority task) before returning control to the preempted task.

Scheduler: Non-Preemptive

- (1) Task is executing, but is interrupted.
- (2) If interrupts are enabled, the CPU vectors to the ISR.
- (3) The ISR handles the event and makes a higher priority task ready to run.
- (4) Upon completion of the ISR the CPU returns control to the interrupted task.
- (5) The task continues from the point it was interrupted.
- (6) When the task completes, it calls a service that the kernel provides to relinquish control of the CPU to another task.
- (7) The kernel sees that a higher priority task is ready to run and performs a context switch to the higher priority task so it can run.



The kernel doesn't know how the task became ready, just that it now is.

Scheduler : Non-Preemptive

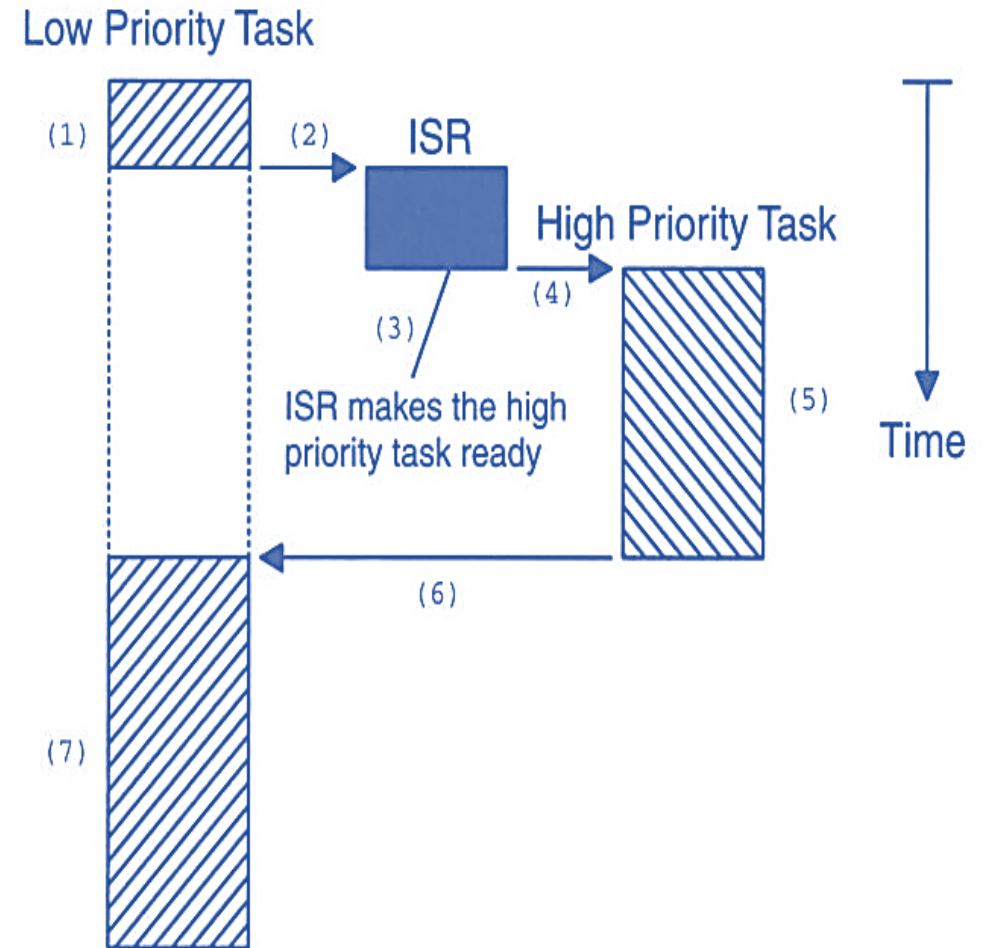
- Also known as cooperative multitasking.
- Non-preemptive kernels require that each task gives up control of the CPU so another task can execute.
- To maintain the illusion of concurrency tasks must give up control of the CPU on a regular basis.
- Asynchronous events are still handled by ISRs.
- An ISR can make a higher priority task ready to run however the ISR returns to the interrupted task, completes the task then runs the higher priority task.
- Interrupt latency is generally low for non-preemptive kernels.
- Also non-preemptive kernels can use non-reentrant functions.
- Non-reentrant functions can be used by each task without the fear of data corruption, as each task must run to completion before giving up the CPU.
- The task-level response of a non-preemptive kernel is much lower than a foreground/background system, as the task level response is defined by the time taken to execute the longest task.
- There is no need to guard shared data (using semaphores) as only one task has access to it at any given time.

Scheduler : Non-Preemptive

- The biggest drawback of a non-preemptive kernel is its responsiveness.
- A higher priority task that has been made ready to run has to wait a long time for the lower priority task to complete before it can get control of the CPU.
- A non-preemptive kernel allows each task to run until it voluntarily gives up control of the CPU.
- An interrupt (ISR) preempts a task. Once completed returns to the preempted task and lets the task complete before running the new task.

Scheduler : Preemptive

- (1) A low priority task is running and is interrupted.
- (2) The CPU services the ISR.
- (3) The ISR handles the event and makes a higher priority task ready to run.
- (4) Upon completion of the ISR a kernel service is invoked that performs a context switch to the higher priority task, instead of returning to the suspended task.
- (5) The higher priority task is run to completion.
- (6) On completion of the higher priority task the kernel realizes that a lower priority task has been suspended and therefore performs a context switch to continue running the suspended task.



Scheduler : Preemptive

- Used when system responsiveness is extremely important.
- Real-time kernels fall into this category.
- The highest priority task that is ready to run is always given control of the CPU.
- When a higher priority task is ready to run the current running task is preempted (suspended) and the higher priority task is run to completion (unless interrupted by an even higher priority task) before returning control to the preempted task.
- When an ISR makes a higher priority task ready, the higher priority task is immediately given control of the CPU and the interrupted task is suspended.
- Using a preemptive kernel you have more control over when an actual task is executed.
- The task-level response time is therefore minimized.
- Application code running under a preemptive kernel should use reentrant code, unless of course mutual-exclusion mechanisms are used.
- Otherwise data corruption will occur if a higher priority task utilizes the same code that is being used by a lower priority task.
- A preemptive kernel will always execute the highest priority task, suspending the lower priority task until the higher priority task has completed.

Scheduler: Round Robin Scheduling

- Also known as *time-slicing*.
- When two or more tasks have the same priority, the kernel allows one task to run for a predetermined amount of time (*quantum*) and then selects another task of the same priority to run.
- See Lecture 3A slide 16

Scheduler : Task Priorities

Task Priorities

A priority is assigned to each task, the higher the priority the more important the task is. With most kernels you are responsible for assigning the priority values.

Static Priorities

Static priorities are assigned to tasks at compile time and do not change during the running of the application.

Dynamic Priorities

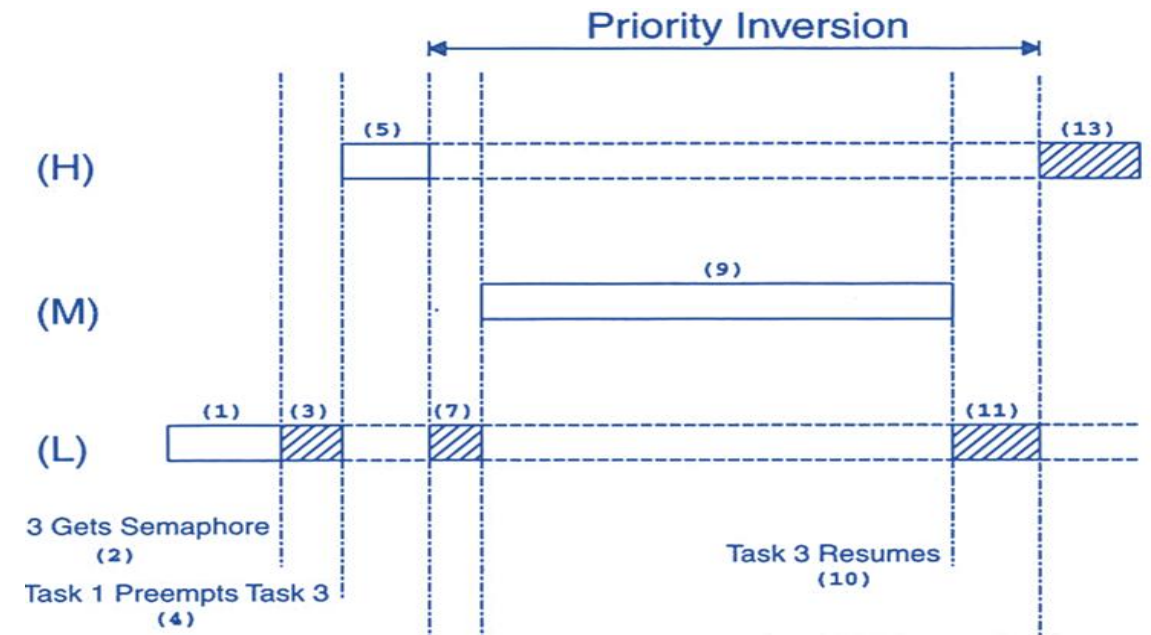
Task priorities are referred to as dynamic if the priority can be changed during the running of the application.

Priority Inversion

Priority inversion is a problem when a task of low priority registers a semaphore over a shared resource, is interrupted to run a higher priority task which also wants access to the shared resource. The higher priority task can not gain access to the resource until the lower priority task releases the semaphore

Scheduler : Priority Inversion Example

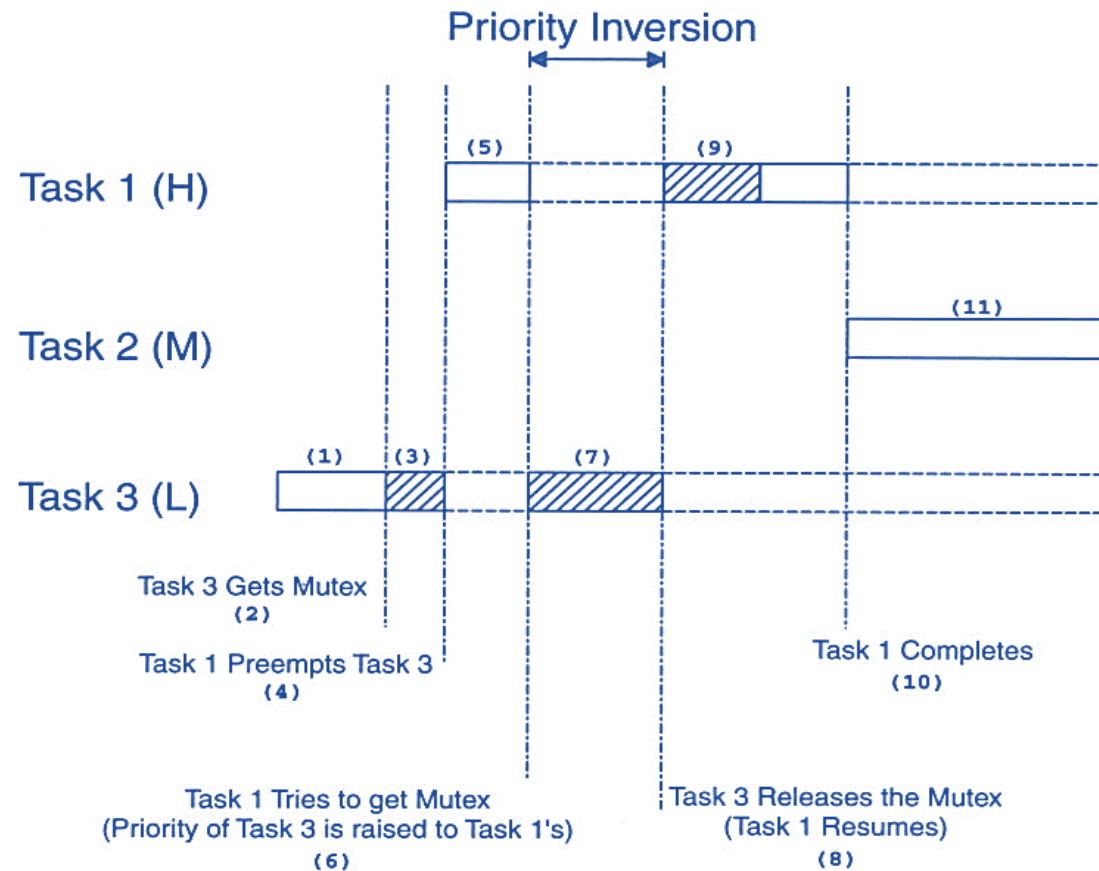
- (1) Task 1 and 2 are waiting for an event to take place. Task 3 is running.
- (2) Task 3 acquires a semaphore.
- (3) Task 3 performs some operation with the resource.
- (4) The event task 1 was waiting for takes place. Task 3 is suspended and task 1 gets control of the CPU.
- (5) Task 1 executes.
- (6) Task 1 attempts to acquire semaphore for the same resource.
- (7) Task 1 is placed on a list of tasks waiting for a semaphore.
- (8) Task 3 resumes.
- (9) The event Task 2 was waiting for occurs, CPU is given to Task 2.
- (10) Task 2 completes and returns control back to Task 3.
- (11) Task 3 finishes with the resource and releases the semaphore.
- (12) At this point the kernel gives control to Task 1.



Scheduler : Priority Inversion

- Due to priority inversion the priority of Task 1 was reduced to priority of Task 3.
- This could have been avoided by raising the priority of Task 3 to that of Task 1, while Task 3 held onto the semaphore.
- Altering a Tasks priority takes time and the kernel needs to be able to alter task priorities during runtime.
- Altering a Tasks priority to prevent this condition from happening is commonly referred to as ***Priority Inheritance***.

Scheduler : Priority Inheritance



OS : Intertask Communications

- There are times in which a task or ISR needs to communicate information to another task.
- There are two possible ways of achieving this:
 Using Global variables, or
 Sending Messages.

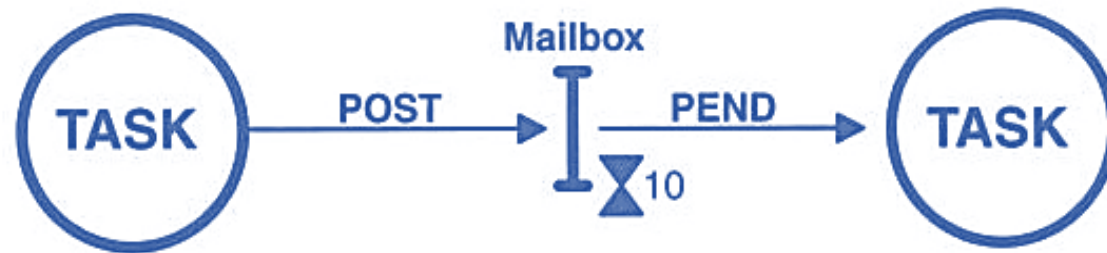
Global Variables

- If global variables are used, care must be made to ensure a task has exclusive access to this variable.
- If an ISR wants exclusive access to a global variable it must ensure that interrupts are disabled when the variable is accessed.
- If a TASK requires exclusive access to a global variable it must ensure that either interrupts are disabled or a semaphore is used.
- Note that data can only be conveyed to an ISR using global variables.
- A task is not aware that a global variables has been changed by an ISR unless a semaphore is used.
- This can be solved using either message mailboxes or message queues.

OS : Intertask Communications

Message Mailboxes

- A message is a kernel service generally the size of a pointer.
- The kernel provides a service that enables a message (pointer) to be deposited into a mailbox by either an ISR or another Task.
- Similarly the kernel provides a service that enables one or more tasks to receive the message through a mailbox.

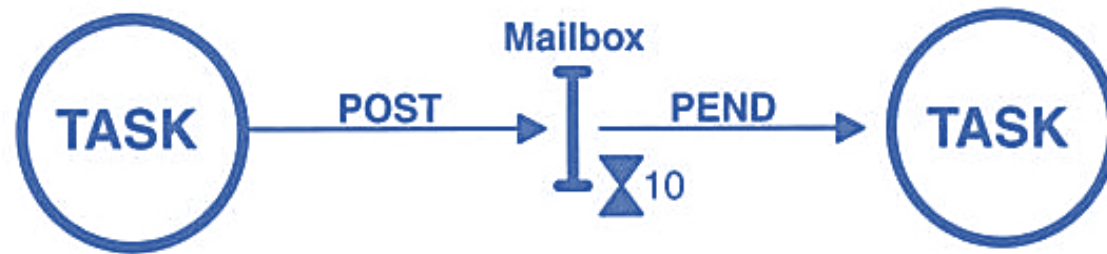


Message Mailbox.

OS : Intertask Communications

Message Mailboxes

- Both the sender and receiver must agree on what the pointer actually points to.
- Each mailbox has a waiting list associated with it. A task desiring a message from an empty mailbox is suspended and placed on the waiting list until the mailbox is no longer empty.
- As with semaphores a timeout can be provided, once exceeded, the requesting task continues and an error code is provided to the calling task outlining what error occurred.

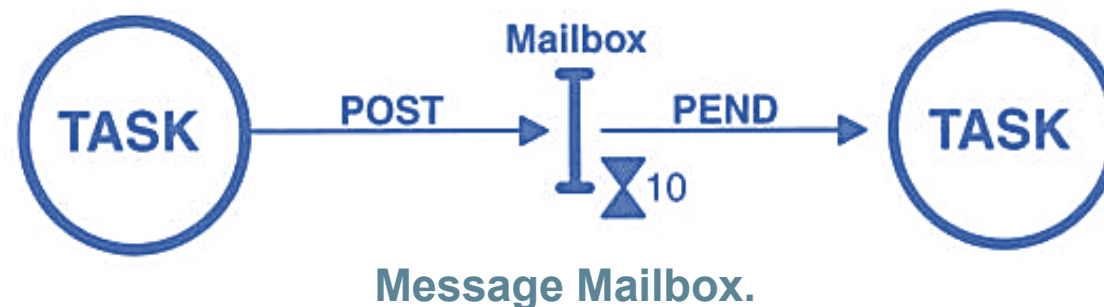


Message Mailbox.

OS : Intertask Communications

Message Mailboxes

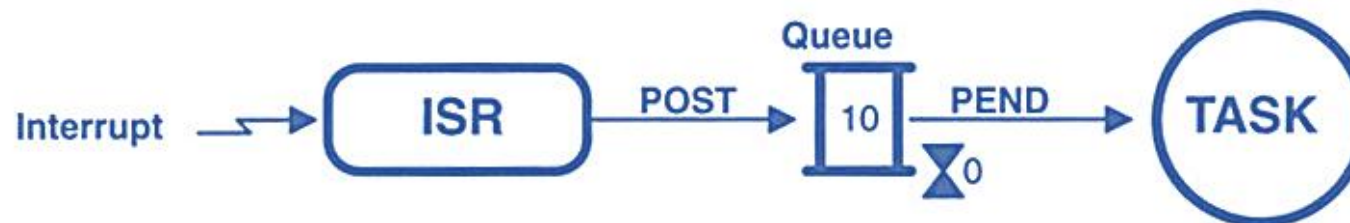
- When one or more tasks are waiting for a message to appear in a mailbox, either the message is given to the highest priority task or the first task that requested the message.
- A message mailbox can simulate binary semaphores.
- Kernels typically provide the following mailbox services:
 - Initialize the contents of a mailbox.
 - Deposit a message into the mailbox (**POST**).
 - Wait for a message to appear (**PEND**).
 - Get a message from the mailbox (**ACCEPT**).



OS : Intertask Communications

Message Queues

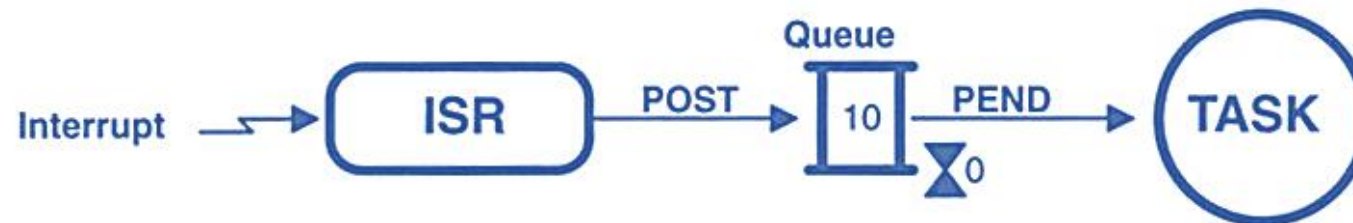
- A message queue is used to send multiple messages to a task.
- A message queue can be thought of as an array of mailboxes.
- As with mailboxes, tasks or ISRs can deposit a message into a message queue via a kernel service.
- Also one or more tasks can read messages posted to a message queue.
- Like mailboxes, both the sender and receiver must agree on what the pointer is pointing to.
- Generally the first message inserted into the queue is the first message extracted from the queue (FIFO).
- Some kernels in fact allow you to also extract messages in a stack-like fashion (Last-In-First-Out).
- Also the message queue has a waiting list and timeout associated with it.



OS : Intertask Communications

Message Queues

- A kernel usually provides the following message queue services:
 - Initialize the queue.
 - Deposit a message onto the queue (POST).
 - Wait for a message to be deposited onto the queue (PEND).
 - Get a message from the queue (ACCEPT).



OS : Special Interrupt

Clock Tick

- A clock tick is a special interrupt that occurs periodically.
- It can be seen as the systems heartbeat.
- The clock tick interrupt allows the kernel to delay a task by a predefined amount of time and provide timeouts for semaphores and message queues.
- All kernels allow task to be delayed by a number of clock ticks. Even though the delay resolution is a single clock tick does not mean its accuracy is one clock tick.

References

- [1] Labrosse, J. J., “MicroC/OS-II: The real-time Kernel”, CMP Books, 2003.
- [2] Deitel, H. M., “Operating Systems”, Addison Wesley, 1990.
- [3] Simon, D. E., “ An Embedded Software Primer”, Addison Wesley, 1999.